GEOWATCH — GEOFENCING, SPATIAL INDEXING & RATE LIMITING

SECTION 1 — GEOFENCING

1. What is geofencing?
Geofencing creates a virtual geographical boundary around an event. GeoWatch uses it to check whether an incident was reported inside the allowed area.

2. How does GeoWatch use geofencing?
GeoWatch calculates the distance between the reported incident GPS coordinates and the event center, and compares it to the allowed event radius.
Event center + radius ↓ Incident GPS coordinates ↓ Calculate distance ↓ Compare with allowed radius ↓ Accept / Reject

3. Where does the geofencing calculation happen?
It is performed on the backend inside **IncidentService.submitIncident()** so the backend remains the secure source of truth, preventing API payload tampering.

4. How is the distance calculated?
GeoWatch uses the Haversine formula to calculate the distance between two points on Earth using their latitude and longitude.

$$a = \sin^2(\Delta\phi/2) + \cos(\phi^1) \cdot \cos(\phi^2) \cdot \sin^2(\Delta\lambda/2)$$
$$c = 2 \cdot \arctan2(\sqrt{a}, \sqrt{(1-a)})$$
$$d = R \cdot c$$

Where: ϕ = latitude, λ = longitude, **R** = Earth's radius (6,371,000 meters), **d** = distance in meters. In GeoWatch, this calculated distance is compared with the event's allowed radius.

5. How does the radius work?
The event has a center coordinate and an allowed radius. If the calculated distance from the event center is within the allowed boundary, the incident passes the geofence; otherwise it is rejected.

6. What is the 30m buffer?
GeoWatch adds a 30-meter tolerance to the configured event radius to account for GPS inaccuracies. For example, if Configured radius = 500m and Buffer = 30m, the Effective boundary is 530m. The 30m buffer is an additional GPS tolerance, not the event radius itself.

SECTION 2 — CUSTOM GRID-BASED SPATIAL INDEX

7. What is a custom grid-based spatial index?
A grid-based spatial index divides the geographical area into small cells so we can find nearby incidents without checking every incident.
GPS coordinate ↓ Find grid cell ↓ Check nearby cells ↓ Get nearby incident points

8. Where is the grid-based spatial index implemented?
File: *DbscanClusteringService.java* | **Class:** *SpatialIndex* (private static class)
It maps incidents to coordinate grid cells based on DBSCAN epsilon search radius using a lookup map (*Map<String, List<Incident>> grid*).

9. Why do we need the grid index?
Without a spatial index, we would need to check all points against every other point, leading to an O(N²) calculation scan. The grid allows checking only the current and 8 adjacent cells (dx, dy from -1 to 1), avoiding slow table scans.
Without grid: Point → many/all points | **With grid:** Point → nearby cells → nearby points

10. How does the grid index help DBSCAN?
DBSCAN needs to query all points within distance *eps* to expand clusters. The grid index acts as a fast neighbor look-up provider, preventing DBSCAN from scanning every single point in the database.
Grid Index ↓ Find nearby candidate points ↓ DBSCAN ↓ Create clusters

SECTION 3 — RATE LIMITING

11. What is rate limiting?
Rate limiting controls how many requests a user can make within a specific time period to prevent API spam.

12. Why do we use rate limiting?
It prevents excessive or spam incident submissions, protecting server and database resources.

13. What is the GeoWatch rate limit?
3 incident reports per 5 minutes.

14. How does the rate limit work?
Incident request ↓ Identify user ↓ Find recent incidents ↓ Count ↓ Compare with limit ↓ Accept / Reject
The backend checks the database for incidents submitted by the same phone number within the last 5 minutes. If 3 or more reports exist, it throws a runtime exception.

15. How do we identify the user?
GeoWatch identifies the user by the phone number sent in the incident creation payload (accessed via *request.getPhoneNumber()*).

16. Why do we use the phone + timestamp database index?
The composite index **idx_incident_phone_timestamp** on columns *(phone_number, timestamp)* ensures the rate limit query is extremely fast and avoids reading the entire database table.

Incident Request ↓ Geofence Check ↓ Rate Limit Check ↓ Save Incident